focus on options available for higher-level BDD in JavaScript, using Gherkin-style scenario definitions.

There are a number of tools available to JavaScript developers that support Gherkin scenario descriptions, including Cucumber-JS (https://github.com/cucumber/cucumber-js), which is probably the best known of the JavaScript BDD libraries, and Yadda (https://github.com/acuminous/yadda), an alternative to Cucumber-JS that allows more flexibility in the scenario wording.

Let's take a closer look at what BDD in JavaScript using Cucumber-JS involves.

6.8.1 Setting up Cucumber-JS

Cucumber-JS is intuitive and easy to set up, and it's straightforward to use. Cucumber-JS relies on Node.js (a JavaScript application platform widely used in modern JavaScript development, found at http://nodejs.org) and npm (the Node.js package manager) to work, so you will need to install Node.js if you want to follow along with this example. Once it's installed, you can install Cucumber-JS by running the following:

```
$ npm install -g cucumber
```

This will install Cucumber-JS globally, so you can call it from any project. To check that it worked, run `cucumber-js -help` from the command line.

6.8.2 Writing feature files in Cucumber-JS

By default, Cucumber will expect the feature files and step definitions to be in specific directories, as illustrated in figure 6.12. Of course, you can override this structure if you want to, but for this example you'll go with the flow and use the defaults.

Much like Cucumber-JVM, Cucumber-JS expects the feature files to be in a directory called features. Cucumber-JS also understands the same Gherkin syntax that you've seen for the other Cucumber-derived BDD tools.

Imagine you're writing a client JavaScript application for the Frequent Flyer website that needs to be able to calculate the points earned for a given trip in real time. You might reuse the following scenario for your client-side business logic:

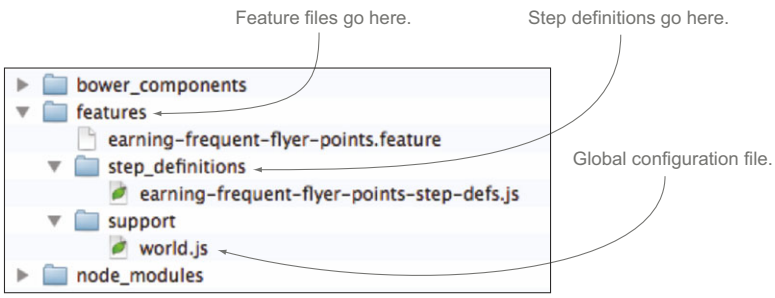


Figure 6.12 The default Cucumber-JS directory conventions